APEX

Android Physical EXtraction

Final Year Project Proposal

Supervisor: Dr. Umer Suleman

Industrial Supervisor: Hamad Riaz

Consultant: Bilal Hashmi

Department of Computer Science | Information Technology University, Lahore | 2026

1. Problem Statement

Physical memory (RAM) on a live Android device is the most forensically valuable and time-sensitive artifact available during a digital investigation. Unlike encrypted persistent storage, RAM contains plaintext data actively in use by the operating system and applications — decrypted messages, session tokens, cryptographic keys, credentials, and process state that would be inaccessible from any other acquisition vector. The moment a device is powered off, this evidence is permanently lost.

The fundamental challenge is that Android's security architecture is explicitly designed to prevent any process from reading physical memory. SELinux mandatory access controls, per-application UID isolation, kernel namespace separation, and restricted /dev/mem access collectively ensure that no userspace process can read memory outside its own allocation. Acquiring a raw physical memory image therefore requires operating at the kernel level, below Android's security enforcement boundary.

Existing approaches to this problem require either an unlocked bootloader or a pre-rooted device — preconditions that are forensically unacceptable. Unlocking a bootloader triggers a factory reset, destroying the very evidence being sought. Commercial forensics platforms such as Cellebrite circumvent this by using kernel-level exploits against known vulnerabilities, but their methodology is entirely closed-source, undocumented, and inaccessible to the research community.

CVE-2019-2215 is a use-after-free vulnerability in the Linux kernel's Binder IPC driver, disclosed by Jann Horn at Google Project Zero in October 2019. It affects Android devices running kernel versions prior to the December 2019 security patch and provides an attacker with arbitrary kernel read/write primitives from an unprivileged userspace process. Existing public research exploits this vulnerability exclusively for privilege escalation — obtaining a root shell or bypassing SELinux. No prior work has used this exploit primitive as a forensic acquisition vector to produce a raw physical memory image from a locked, unmodified Android device.

APEX (Android Physical EXtraction) addresses this gap. It uses CVE-2019-2215 to achieve kernel-level code execution on a locked Android device without modifying the bootloader or requiring prior root access, then deploys a physical memory acquisition payload from within kernel context to produce a complete raw RAM image for forensic analysis. This is the first open-source, academically documented implementation of exploit-assisted live memory forensics on Android.

2. Objectives

- Analyse CVE-2019-2215 at the kernel level — understand the use-after-free primitive in the Binder driver, the exploit chain from UAF to arbitrary kernel read/write, and the specific kernel structures and offsets required for reliable exploitation on ARM targets.
- Implement a working exploit for CVE-2019-2215 targeting ARM Android devices running a vulnerable kernel, validated first on a QEMU ARM emulation environment and subsequently on real hardware.
- Design and implement a kernel-level RAM acquisition payload in C that executes once kernel code execution is achieved — walking the physical memory map via the kernel iomem resource tree and producing a complete raw memory image.
- Develop a host-side acquisition controller in C that orchestrates the full chain: pushing the exploit binary via ADB, triggering execution, and pulling the resulting raw memory image to the host machine with acquisition metadata and integrity verification.
- Perform forensic analysis of acquired memory images using the Volatility Framework, documenting recoverable artifact classes including decrypted application data, process state, network connections, and cryptographic key material.
- Formally document the complete attack and acquisition chain, compare the security posture of vulnerable versus patched Android versions, and produce a research paper suitable for submission to a peer-reviewed digital forensics venue.

3. Related Work

APEX sits at the intersection of two research areas: Android kernel exploitation and mobile memory forensics. Prior work in each area informs the project but no existing work combines them in the manner APEX proposes.

CVE-2019-2215 and Binder Exploitation

The vulnerability was discovered and disclosed by Jann Horn (Google Project Zero, 2019) with a detailed technical writeup documenting the race condition in the Binder driver's BINDER_THREAD_EXIT handling, the use-after-free on the binder_thread structure, and the path to kernel read/write primitives via pipe buffer manipulation. Multiple public PoC implementations exist (Kangtastic, timwr, DimitriFourny on GitHub) targeting specific devices including Pixel 2 and various Samsung models. All existing implementations use the exploit for privilege escalation only — obtaining a root shell or bypassing SELinux enforcement. None implement a forensic acquisition payload.

Android Live Memory Forensics

LiME (Linux Memory Extractor) by Joe Sylve (2012) established the foundational methodology for kernel module-based physical memory acquisition on Linux and Android. LiME requires loading a kernel module via insmod, which in turn requires either root access or an unlocked bootloader — preconditions that are forensically unacceptable on a seized device. Sylve et al. further documented recoverable forensic artifacts from Android RAM dumps in "Acquisition and Analysis of Volatile Memory from Android Devices" (Digital Investigation, 2012), providing the artifact analysis baseline APEX builds upon.

Commercial Exploit-Assisted Acquisition

Cellebrite UFED and similar commercial platforms are known to use kernel exploits for physical acquisition on locked devices, as evidenced by leaked internal documents and

independent reverse engineering by the security research community. However, their methodology is entirely proprietary and undocumented. APEX produces the first open-source, academically documented implementation of this class of technique.

Android Kernel Hardening

Android kernels post-2020 incorporate significant exploit mitigations including CFI (Control Flow Integrity), ShadowCallStack, KASLR, and BTI (Branch Target Identification) on ARM. These mitigations are documented in the Android security documentation and in academic work on kernel exploit mitigations. APEX's paper component will formally analyse why these mitigations defeat the CVE-2019-2215 exploit chain on modern kernels, providing a clear boundary between vulnerable and hardened device generations.

4. Tools and Technologies

Development Environment

- Arch Linux — primary development platform
- Android NDK (aarch64-linux-android toolchain) — cross-compilation of exploit and acquisition payload for ARM Android targets
- GCC arm-none-linux-gnueabihf — kernel module cross-compilation
- QEMU with ARM system emulation — initial exploit development and validation environment
- Android Studio AVD — supplementary emulation for ADB workflow testing

Exploit Development and Analysis

- Ghidra — ARM binary analysis and kernel structure offset identification
- Wireshark (usbmon) — USB traffic capture for ADB protocol analysis
- GDB with gdbserver — kernel-level debugging on target device
- /proc/kallsyms, /proc/iomem — kernel symbol and memory map enumeration on target

Core Implementation

- C — exploit binary, kernel acquisition payload, host-side acquisition controller
- ADB (Android Debug Bridge) — host-to-device communication channel for exploit delivery and image retrieval
- Linux Kernel Module API — iomem resource tree iteration, physical memory mapping, procfs interface

Forensic Analysis

- Volatility Framework — forensic analysis of raw memory images
- strings, binwalk — supplementary artifact extraction

Target Hardware

- Android device running kernel version vulnerable to CVE-2019-2215 (pre-December 2019 security patch)
- ARM architecture (aarch64), USB debugging enabled — no bootloader unlock required
- QEMU ARM image with vulnerable kernel for initial development and midterm demonstration

5. Initial Plan of Work

The project is structured across two semesters in five sequential phases. Each phase has defined deliverables that gate progress to the next phase.

Phase 1 (Weeks 1-4): Foundations

- Complete Android OS architecture study — boot chain, partition layout, SELinux model, FBE encryption, binder IPC
- Complete C systems programming fundamentals — kernel memory allocators, pointer arithmetic, manual memory management
- Set up QEMU ARM emulation environment on Arch Linux with a vulnerable Android kernel image
- Confirm ADB connectivity to QEMU instance and validate kernel version is pre-December 2019 patch
- Deliverable: working QEMU ARM Android environment with confirmed vulnerable kernel

Phase 2 (Weeks 5-9): Exploit Understanding and Validation

- Study Jann Horn's CVE-2019-2215 writeup in full — binder UAF, pipe buffer primitive, kernel r/w chain
- Study existing public PoC implementations (kangtastic, timwr) — understand offset identification and ARM-specific exploitation
- Compile and run existing PoC on QEMU environment — confirm kernel code execution achieved
- Identify and document all device-specific offsets required for the exploit chain
- Deliverable: working CVE-2019-2215 exploit achieving kernel execution on QEMU — midterm demonstration ready

Phase 3 (Weeks 10-14): Acquisition Payload Development

- Design kernel-level RAM acquisition payload — physical memory range enumeration via iomem, raw read, output interface
- Implement payload as position-independent code injectable via the exploit's kernel write primitive
- Integrate payload with exploit chain — exploit achieves kernel execution, payload executes, memory image produced
- Validate memory image integrity against known process artifacts visible via /proc
- Deliverable: end-to-end chain from exploit trigger to raw memory image on QEMU

Phase 4 (Weeks 15-19): Host Controller, Real Hardware, and Artifact Analysis

- Implement host-side C controller — ADB push, exploit trigger, image pull, metadata, SHA256 integrity hash
- Port and validate exploit chain on real ARM Android hardware with confirmed vulnerable kernel
- Run Volatility against acquired images — document recoverable artifacts: process list, network state, decrypted app data, key material

- Deliverable: complete working tool on real hardware with documented forensic artifact recovery

Phase 5 (Weeks 20-24): Documentation, Paper, and Release

- Write formal research paper: exploit chain analysis, acquisition methodology, artifact findings, comparison with patched Android versions and their mitigations
- Package APEX as open-source release on GitHub with build instructions, usage documentation, and supported device list
- Prepare FYP final report and defence presentation
- Target submission to DFRWS, IEEE S&P workshop, or equivalent digital forensics / security venue
- Deliverable: final report, open-source release, conference paper submission

Research Contribution and Novel Angle

CVE-2019-2215 has been used exclusively in public research for privilege escalation — obtaining a root shell or bypassing SELinux. APEX is the first work to use this exploit primitive as a forensic acquisition vector, extending the kernel code execution primitive into a complete RAM acquisition chain that produces a raw physical memory image from a locked, bootloader-intact Android device. This novel application of a known exploit to a forensic use case, combined with formal artifact analysis and an open-source tool release, constitutes the original research contribution of this project.